



MANIPAL INSTITUTE OF TECHNOLOGY

BENGALURU

(A constituent unit of MAHE, Manipal)

School of Computer Engineering

Database Systems Lab [CSS 2212]

Mini Project Report

Rakt-Kosh

Real Time Blood Inventory & Donor Network

Project Team		
Registration No.	Name of the Student	Sem - Branch - Section
245805136	Swasti Gupta	IV - CSE - A
245805029	Arisha Jaiswal	IV - CSE - A
245805398	Manya Vishwanth	IV - CSE - A
245805104	Dhruvishi Amin	IV - CSE - A

Under the guidance of

Dr Soumyashree

Assistant Professor Senior Scale
School of Computer Engineering
Manipal Institute of Technology



MANIPAL INSTITUTE OF TECHNOLOGY

BENGALURU

(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

MIT Bengaluru

09-04-2026

CERTIFICATE

This is to certify that the Mini Project titled **Rakt Kosh: A real-time blood inventory & donor network** is a report of the bona fide work carried out by **Swasti Gupta, Arisha Jaiswal, Manya Vishwanth & Dhruvishi Amin (Register No(s). 245805136, 245805029, 245805398, 245805104)**, submitted in partial fulfillment of the requirements of the course **CSS_2212 – Database Systems Lab** of the **IV Semester**, in the School of Computer Engineering, Manipal Institute of Technology, Bengaluru, Karnataka (a Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025–2026.

Dr Soumyashree

Assistant Professor Senior Scale

SOCE, M.I.T, BENGALURU

Dr. Jayitha Saha

Program Coordinator

Associate Professor

SOCE, M.I.T, BENGALURU

ABSTRACT

In the modern Indian healthcare landscape, where the **demand for blood often outstrips supply by over 1 million units annually**, the timely availability of blood products is a critical factor in patient survival. Despite the growth of the healthcare sector, India's blood banking system remains highly fragmented, leading to life-threatening shortages during emergencies and high wastage due to lack of coordination between regional centers. As urban centers like Delhi, Mumbai, and Bengaluru expand, the need for a **centralized, location-aware system that connects the diverse donor population with local blood banks** has become an urgent necessity. The primary objective of this project is to develop a robust Database Management System (DBMS) tailored for the Indian context to streamline inventory tracking, automate expiration protocols, and utilize geospatial data to bridge the gap between supply and demand.

The methodology adopted for this project follows a full-stack development lifecycle centered around a relational database architecture. **MySQL** was utilized to manage complex relations between blood banks, donors, and inventory, employing **spatial functions** for proximity-based searching across Indian districts.. The backend logic was constructed using **FastAPI** (Python), which handles the geocoding of Indian city names and addresses into coordinates and manages automated business rules, such as the 42-day inventory expiry and the 56-day donor eligibility window in accordance with the National Blood Transfusion Council (NBTC) guidelines. The frontend was developed using **React.js**, providing a dual-interface dashboard for both administrative blood bank tasks and user-centric donor/recipient services.

The results of this implementation demonstrate a significant improvement in resource visibility across diverse geographical areas. By integrating automated notifications, the system ensures that recipients across various states are prioritized the moment matching blood types are entered into the database. The geospatial search functionality successfully allows users to locate the nearest blood drives and available units within their specific municipality with high precision, significantly reducing the "search time" during critical medical emergencies. Furthermore, the automated status-tracking system effectively eliminated the risk of issuing expired blood, ensuring that only viable units are displayed to users based on real-time collection data from participating Indian blood banks.

In conclusion, this project provides a scalable and reliable solution to the logistical challenges faced by the Indian blood banking infrastructure. By automating the tracking of donor eligibility and inventory shelf-life, the system minimizes human error and optimizes the lifecycle of every donation in a country with high seasonal demands. The integration of location-based services represents a forward-thinking approach to public health, transforming blood donation from a static resource into a dynamic, responsive network suitable for the Indian demographic.

Software Tools/Packages: MySQL, Python (FastAPI, SQLAlchemy, Geopy), React.js, Node.js, and Vite.

Table of Contents

	Page No
Acknowledgement	i
Abstract	ii
List Of Tables	iii
List Of Figures	iv
Chapter 1 Introduction	
1.1 Overview	1
1.2 Problem Definition	1
1.3 Purpose & Objectives	2
1.4 Scope of the System	2
Chapter 2 Literature Survey and Gaps	
2.1 Literature Survey	3
2.2 Existing Systems/ Similar Applications	3
2.3 Limitations of Current Systems	4
2.4 Research Gaps	4
2.5 Need for Rakt-Kosh	4
Chapter 3 Problem Statement and Objectives	
3.1 Problem Statement	6
3.2 Objectives	6
Chapter 4 System Architecture	
4.1 Presentation Layer (Frontend)	7
4.2 Application Layer (Backend)	7
4.3 Data Layer (Database)	8
4.4 System Interaction and Data Flow	8
Chapter 5 Relational Data Model and Functionalities	
5.1 Database Entities and Attributes	9
5.2 Normalization	10
5.3 E-R Diagram	11
Chapter 6 Technologies Used, Front End Design and Backend Logic	
6.1 Technologies Used	12

6.2	Front End Design	12
6.3	Backend Logic	13
6.4	Sample Code/Queries	13
6.5	Screens/Forms	16
Chapter 7	Data Flow and Results and Discussions	
7.1	Flow of Data Across Different Modules	20
7.2	Output Screenshots and Query Logic	22
7.3	Explanation of Queries/Results	24
7.4	Performance or Observations	25
Chapter 8	Conclusions and Future Scope	
8.1	Conclusions	27
8.2	Future Scope	27
References		

LIST OF TABLES

Table No	Table Title	Page No
1	Database Entities and Key Descriptions	10
2	Technologies and Tools Used	12
3	User Registration Data Flow	20
4	User Login Data Flow	20
5	Find Nearest Drives Data Flow	21
6	Blood Request Data Flow	21
7	Add Inventory Data Flow	21
8	Schedule Drive Data Flow	22
9	Background Automation Data Flow	22

LIST OF FIGURES

Figure No	Figure Title	Page No
1	System 56-Day Donor Recuperation Window	5
2	Entity-Relationship Diagram	11
3	Dynamic Red Wave Homepage	16
4	Dual Registration Portal	16
5	Patient Dashboard - Find Blood Results	17
6	Patient Dashboard - Find Donation Drives	17
7	Patient Dashboard - Request Blood	18
8	Admin Dashboard - Live Pending Requests	18
9	Admin Dashboard - Live Blood Inventory	19
10	Admin Dashboard - Log Blood Units	19
11	Admin Dashboard - Schedule Donation Drive	19

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

Rakt-Kosh is a centralized, real-time web application designed to bridge the gap between blood donors, medical institutions, and recipients. Built using a modern full-stack architecture—comprising **React** for the frontend, **FastAPI** for the backend, and **MySQL** for relational data management—the platform serves as a digital ecosystem for blood supply chain management. By integrating geospatial services, the system allows for the precise tracking of blood inventory and the discovery of nearby donation drives, transforming a traditionally fragmented process into a streamlined, data-driven operation.

1.2 Problem Definition

In critical medical emergencies, time is the most significant factor in determining patient outcomes. Currently, many blood banks and hospitals in urban centers like Bengaluru operate in isolation, maintaining manual or localized records. This leads to several critical inefficiencies:

- **Information Asymmetry:** Patients and their families often have to contact multiple facilities individually to check for specific blood types, losing vital time.
- **Manual Inventory Tracking:** Without real-time updates, a blood bank may appear to have stock on paper while being physically depleted.
- **Donor Retention Gaps:** There is often no automated mechanism to track a donor's last donation date, leading to missed opportunities for eligible donors to contribute again.
- **Geographical Barriers:** Potential donors are often unaware of donation drives happening in their immediate vicinity, resulting in lower participation rates.

1.3 Purpose and Objectives

The primary purpose of Rakt-Kosh is to digitize the blood donation lifecycle to ensure that life-saving resources are always accessible and traceable. The specific objectives include:

- **Centralization of Data:** To provide a single source of truth for blood availability across multiple registered banks.
- **Geospatial Optimization:** To implement location-based search functionality that identifies the nearest donation drives using spherical distance calculations.
- **Automated Eligibility Monitoring:** To utilize background scheduling tasks that identify and notify donors who have completed the mandatory 56-day recovery period.

- **Secure Authentication:** To ensure the integrity of user data and medical records through industry-standard encryption and hashing protocols.
- **Efficient Request Management:** To provide a dedicated interface for patients to submit emergency blood requirements, which are then logged and prioritized by the system.

1.4 Scope of the System

The system is designed to handle the core operational requirements of a blood donation network, specifically focusing on the following functional areas:

- **User Management:** Secure registration and login for donors and recipients, including the storage of geographical coordinates (latitude and longitude) for localized services.
- **Inventory Control:** Comprehensive tracking of blood units, categorized by blood group and collection date, with status updates ranging from "Available" to "Requested."
- **Donation Drive Coordination:** A specialized module for administrators to schedule and publicize donation drives, complete with automated geocoding for city-based location tagging.
- **Distance-Based Searching:** An intelligent search engine that calculates the proximity of users to donation events, sorting results by distance to provide the most relevant options first.
- **Eligibility Logic:** Backend automation that monitors the **LastDonationDate** for all registered users and updates their eligibility status in real-time.

CHAPTER 2

LITERATURE SURVEY AND GAPS

2.1 Literature Survey

Blood bank management has evolved from manual ledger-based record-keeping to centralized digital repositories. In India, a primary government initiative was launched in 2016 which serves as a centralized interface for blood inventory across the country. Research in this field has traditionally focused on two areas:

1. **Inventory Management:** Algorithms designed to reduce "wastage" (expiration) vs. "shortage" (unmet demand).
2. **Donor Retention:** Systems that track donor history to encourage repeat donations.

Recent studies have explored the integration of **Cloud Computing** and **Mobile Applications** to increase the reach of these systems, especially in rural areas where the gap between blood demand and supply is most acute.

2.2 Existing Systems or Similar Applications

- **e-RaktKosh (India):** A centralized blood bank management system that provides a portal for citizens to find blood availability. It standardizes the workflow of blood banks across many states.
- **Friends2Support:** One of the world's largest voluntary donor apps, focusing primarily on connecting a seeker directly with a donor via phone numbers.
- **Red Cross Blood Donor App (Global):** A highly advanced system used in the US that allows donors to schedule appointments, track their "Blood Journey," and receive rewards.

2.3 Limitations of Current Systems

Despite the existence of the platforms mentioned above, several critical limitations persist:

- **Stale Data:** Many existing portals rely on manual updates from hospital staff. If a staff member forgets to update the portal after a unit is used, the system shows "available" when it is actually "out of stock," causing wasted time for families in emergencies.
- **Lack of Proactive Notification:** Most systems are "pull-based," meaning the user must keep checking the site. There is a lack of "push-based" systems that notify a desperate seeker the exact second a matching unit becomes available.
- **Manual Expiry Tracking:** While some systems track dates, few integrate a "hard-stop" logic that automatically removes expired units from the user-view, leading to potential confusion during urgent searches.
- **Geospatial Inefficiency:** Many apps list blood banks alphabetically or by district, but do not provide precise "nearest-to-me" sorting based on the user's current GPS or city-specific coordinates.

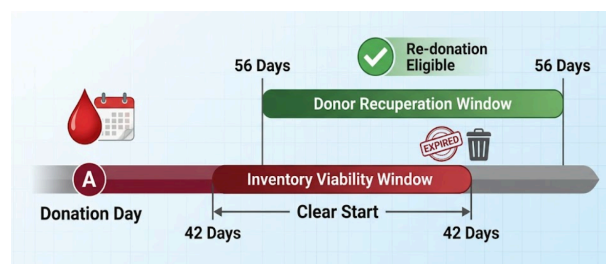
2.4 Research Gaps

The primary gap identified in current literature is the **lack of an integrated, real-time feedback loop between inventory and urgent requests**. Current systems treat inventory and user requests as two separate silos. There is also a significant gap in the automation of "Eligibility Window" notifications in the Indian context, where donors are often unaware of exactly when their 56-day recovery period has ended.

2.5 Need for Rakt-Kosh

The proposed system, Rakt-Kosh, is designed to fill these gaps by:

- **Automating the Expiry Logic:** Using a 42-day auto-expiry filter to ensure medical safety and data integrity.
- **Centralizing Urgent Requests:** Providing a dedicated, live dashboard where "Pending Requests" are instantly visible to hospital administrators, sorted by geographic proximity.
- **Advanced Geocoding:** Allowing users to search by city and receiving results sorted by mathematical distance, which is crucial in high-traffic Indian urban environments.
- **Donor Lifecycle Management:** Automating the 56-day notification to convert one-time donors into a sustainable, repeating donor base.



CHAPTER 3

PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

India faces severe blood shortages and high wastage rates due to decentralized, manual inventory tracking and a lack of real-time communication between donors, patients, and hospitals. Current systems act as static record-keepers rather than dynamic networks; they fail to provide real-time, location-based searches for the nearest available blood units or donation drives. This geographic inefficiency and information asymmetry lead to critical delays during medical emergencies, while simultaneously resulting in the expiration of viable blood units at isolated facilities.

3.2 Objectives

The primary objective of this project is to architect a robust, full-stack Database Management System (DBMS) that digitizes and automates the blood donation lifecycle. The specific objectives include:

1. **Centralized Real-Time Tracking:** Develop a dynamic relational database to track blood inventory across multiple hospitals, ensuring users see accurate, live availability.
2. **Role-Based Access Control (RBAC):** Implement a dual-portal interface separating secure Patient/Donor capabilities from Hospital/Blood Bank administrative tools using client-side vaults.
3. **Geo-Spatial Integration:** Utilize MySQL spatial functions (`ST_Distance_Sphere`) and the Geopy API to allow patients to find the closest available blood units and donation drives calculated in precise kilometers.
4. **Automated System Logic:** Engineer automated backend background tasks (using APScheduler) to enforce the 42-day blood expiry protocol and the 56-day donor recuperation notification system.
5. **Interactive Client-Side Filtering:** Provide a seamless frontend user experience (React.js) that allows administrators to instantly filter inventory without redundant server requests.

CHAPTER 4

SYSTEM ARCHITECTURE

The architecture of Rakt-Kosh is designed around a **Three-Tier Architecture** model. This structure ensures a clean separation of concerns, where the user interface, the application logic, and the data storage are managed independently, facilitating easier debugging, scalability, and maintenance.

4.1 Presentation Layer (Frontend)

The presentation layer is the entry point for all users, including donors, recipients, and administrators.

- **Technology:** Developed using **React and Vite**, a declarative JavaScript library for building user interfaces.
- **Functionality:** This layer manages the state of the application, handles form submissions for registration and blood requests, and displays real-time data fetched from the backend. It uses asynchronous API calls to ensure that the interface remains responsive while communicating with the server.

4.2 Application Layer (Backend)

The application layer acts as the intermediary, processing requests from the frontend and performing the necessary computations before interacting with the database.

- **Technology:** Built with **FastAPI**, a high-performance Python framework known for its speed and native support for asynchronous programming.
- **Core Components:**
 - **Authentication Engine:** Utilizes the **Passlib** library with the **bcrypt** algorithm to securely hash and verify user passwords, ensuring that plain-text credentials are never stored.
 - **Geospatial Service:** Integrates the **Geopy** library and **Nominatim API** to convert human-readable city names into geographic coordinates (Latitude and Longitude).
 - **Automation Scheduler:** Implements a background task runner that periodically scans the database to monitor donor eligibility based on the 56-day recovery protocol.
 - **API Routing:** Defines RESTful endpoints for user management, inventory updates, and proximity-based searching.

4.3 Data Layer (Database)

The data layer is responsible for the persistent storage of all platform information. It is managed by MySQL, a relational database management system that ensures data integrity

through foreign key constraints and structured schemas. The schema is organized into **five** primary tables:

- **users:** Centralizes donor and recipient profiles, storing personal details, hashed credentials, and geographical coordinates for location-based services.
- **bloodbanks:** Acts as the administrative backbone for hospitals, storing facility credentials, contact information, and exact coordinates for geo-spatial matching.
- **bloodinventory:** Tracks individual units of blood, their respective blood groups, collection dates, and current status (e.g., "Available"), linked directly to the collecting blood bank.
- **donationdrives:** Stores information about upcoming blood donation events, including scheduling data and precise coordinates for the nearest-drive locator.
- **bloodrequests:** Manages the demand side of the network, logging urgent patient requirements and tracking the progress of fulfillment.
-

4.4 System Interaction and Data Flow

The interaction between these layers follows a structured workflow:

1. **Request:** A user initiates an action, such as searching for the nearest donation drive, by providing a city name in the React frontend.
2. **Processing:** The frontend sends a GET request to the FastAPI backend. The backend geocodes the city name to retrieve its coordinates.
3. **Query:** The backend executes a specialized SQL query using the `ST_Distance_Sphere` function to calculate the distance between the user's coordinates and the drives stored in the MySQL database.
4. **Response:** The database returns the sorted results to the backend, which then transmits the data back to the frontend in JSON format for the user to view.

CHAPTER 5

RELATIONAL DATA MODEL AND FUNCTIONALITIES

5.1 Database Entities & Attributes

a. Users Table

- i. Primary Key - UserID
- ii. Attributes
 - 1. Full Name
 - 2. Email (UNIQUE)
 - 3. PasswordHash
 - 4. PhoneNumber
 - 5. BloodGroup
 - 6. Latitude
 - 7. Longitude
 - 8. LastDonationDate
 - 9. City

b. BloodBanks Table

- i. Primary Key- BankID
- ii. Attributes
 - 1. Name
 - 2. GovtRegistrationNo (UNIQUE)
 - 3. ContactEmail
 - 4. ContactPhone
 - 5. Latitude
 - 6. Longitude
 - 7. Address
 - 8. PasswordHash
 - 9. City

c. BloodInventory Table

- i. Primary Key - BloodBagID
- ii. Foreign Key - BankID (References BloodBanks)
- iii. Attributes
 - 1. BloodGroup
 - 2. CollectionDate
 - 3. Status (Available, Reserved, Used, Expired)

d. DonationDrives Table

- i. Primary Key- DriveID
- ii. Foreign Key - BankID (References BloodBanks)
- iii. Attributes
 - 1. DriveName

2. DriveDate
3. StartTime
4. EndTime
5. Latitude
6. Longitude
7. Address
8. City

e. BloodRequests Table

- i. Primary Key - RequestsID
- ii. Foreign Key - UserID (References Users), BankId (References BloodBanks)
- iii. Attributes
 1. DonationDate
 2. Quantity
 3. Hemoglobin Level

Table Name	Primary Key	Foreign Key	Description
Users	UserID	None	User Entity
BloodBanks	BankID	None	Bank Entity
BloodInventory	BloodBagID	BankID	Links the blood unit to its storage bank
DonationDrives	DriveID	BankID	Links a drive to the organizing bank
BloodRequests	RequestID	UserID	Links request to the seeker

5.2 Normalization

a. First Normal Form (1NF)

- i. *Rule: Each table cell must contain a single (atomic) value, and each record must be unique.*
- ii. Application: In Users and BloodBanks, fields like PhoneNumber and Email are unique. There are no multi-valued attributes (e.g., we don't store multiple phone numbers in one string). All entries in BloodInventory are unique bags identified by BloodBagID.

b. Second Normal Form (2NF)

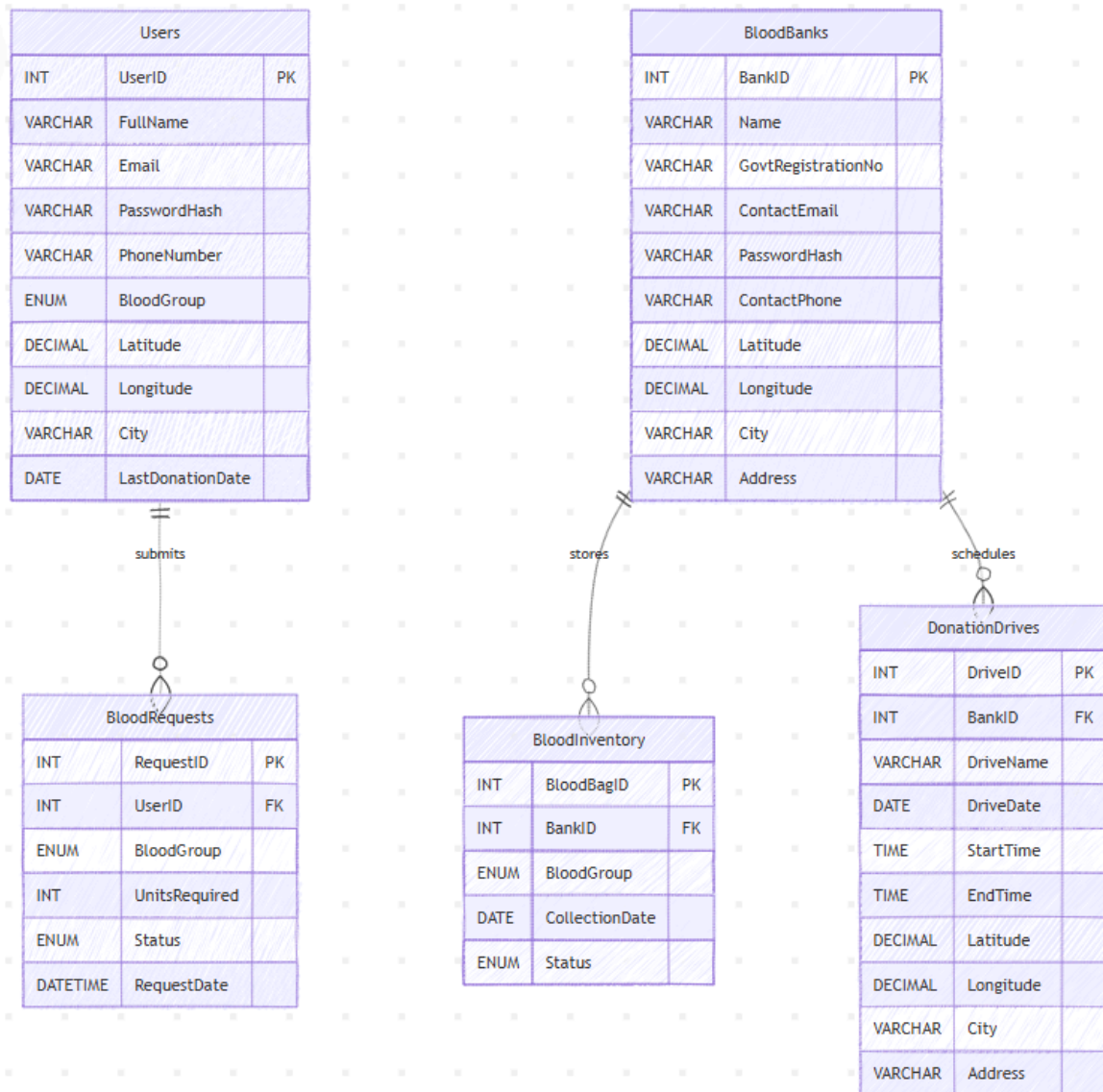
- i. *Rule: Must be in 1NF and all non-key attributes must be fully functionally dependent on the Primary Key.*

- ii. Application: In DonationDrives, the DriveName, StartTime, and Address depend entirely on the DriveID. There is no partial dependency where an attribute depends on only part of a composite key, as we use single-attribute surrogate Primary Keys (INT AUTO_INCREMENT).

c. Third Normal Form (3NF)

- i. Rule: Must be in 2NF and have no transitive dependencies (non-key attributes depending on other non-key attributes).
- ii. Application: In BloodRequests, UnitsRequired and BloodGroup depend on RequestID. In BloodBanks, the Address depends on BankID. No non-key attribute relies on another non-key attribute to exist.

5.3 E-R Diagram



CHAPTER 6

TECHNOLOGIES USED, FRONT END DESIGN AND BACKEND LOGIC

6.1 Technologies Used

Category	Technology / Tool
Frontend Framework	React 19 with Vite build tooling
Routing	React Router DOM v7 (client-side SPA routing)
State Management	React Context API with useState / useContext hooks
Styling	Custom CSS with CSS custom properties (no UI library)
Backend Framework	Python – FastAPI
Database	MySQL with spatial function support (ST_Distance_Sphere)
ORM / DB Access	SQLAlchemy engine with raw parameterised SQL
Request Validation	Pydantic data models
Geocoding	Geopy / Nominatim (OpenStreetMap)
Authentication	Passlib with bcrypt password hashing
Scheduling	APScheduler (background cron-style tasks)
API Protocol	RESTful JSON over HTTP with CORS middleware

6.2 Front End Design

The interface is designed as a Single-Page Application (SPA) with a persistent navigation bar for easy access to its primary routes. The visual identity uses a professional "medical theme" (red-and-white) and incorporates interactive elements such as:

- **Dynamic Visual Interface:** Features a minimalist, CSS-animated fluid wave motif reflecting the application's core theme, prioritizing immediate user action (Registration/Requests) over cluttered dashboards.
- **Badge Grids:** For intuitive, single-click selection of blood groups (A±, B±, AB±, O±) and instant table filtering.
- **Role-Based Dedicated Views:** Used in the Admin Dashboard to securely separate inventory management, event scheduling, and live requests into distinct, authenticated pages.
- **Feedback Loops:** Implementation of loading states and contextual error/success messages.

6.3 Backend Logic

The **FastAPI** backend provides a robust infrastructure for blood donation management by integrating Pydantic for rigorous input validation and **MySQL** for geospatial data processing. The system handles user lifecycle management through a registration process that geocodes locations via the Nominatim API and secures credentials with **bcrypt**, while the login flow utilizes a simplified ID-based session model for the MVP. Its most advanced feature is a proximity engine that uses the **ST_Distance_Sphere** function to calculate great-circle distances, allowing users to find the ten nearest donation drives based on their current city.

Beyond location services, the platform manages logistical operations by allowing administrators to schedule new drives, track available blood inventory, and view active patient requests instantly sorted by geographic proximity to the hospital. To ensure long-term donor engagement and safety, the system incorporates an **APScheduler** background task that monitors the standard 56-day whole blood donation interval. This automation identifies eligible donors and triggers simulated notifications, creating a proactive ecosystem that balances immediate medical needs with sustainable donor health.

6.4 Sample Code/Queries

Database – Table Creation (Users)

The Users table stores donor and patient profiles along with their geocoded coordinates. The email and phone number fields carry unique constraints to prevent duplicate registrations.

```
CREATE TABLE Users (  
  UserID      INT AUTO_INCREMENT PRIMARY KEY,  
  FullName    VARCHAR(100) NOT NULL,  
  Email       VARCHAR(100) NOT NULL UNIQUE,
```

```

PasswordHash VARCHAR(255) NOT NULL,
PhoneNumber VARCHAR(15) NOT NULL UNIQUE,
BloodGroup ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,
Latitude DECIMAL(10,7),
Longitude DECIMAL(10,7), City VARCHAR(100) NOT NULL,
LastDonationDate DATE DEFAULT NULL
);

```

Database – Blood Inventory Table

Each row in BloodInventory represents a single collected blood unit. The Status column tracks whether the unit is still available, has been issued, or has expired.

```

CREATE TABLE BloodInventory (
  UnitID INT AUTO_INCREMENT PRIMARY KEY,
  BankID INT NOT NULL,
  BloodGroup ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,
  CollectionDate DATE NOT NULL,
  Status ENUM('Available', 'Issued', 'Expired') DEFAULT 'Available',
  FOREIGN KEY (BankID) REFERENCES BloodBanks(BankID)
);

```

Database – Donation Drives Table

Donation drives store the event’s schedule alongside geocoded coordinates, enabling the proximity search endpoint to rank them by distance from the user.

```

CREATE TABLE DonationDrives (
  DriveID INT AUTO_INCREMENT PRIMARY KEY,
  BankID INT NOT NULL,
  DriveName VARCHAR(150) NOT NULL,
  DriveDate DATE NOT NULL,
  StartTime TIME NOT NULL,
  EndTime TIME NOT NULL,
  Latitude DECIMAL(10,7),
  Longitude DECIMAL(10,7), City VARCHAR(100) NOT NULL,
  Address VARCHAR(255),
  FOREIGN KEY (BankID) REFERENCES BloodBanks(BankID)
);

```

Database – Blood Banks Table

The BloodBanks table acts as the administrative backbone for hospitals, storing facility credentials, contact information, and exact coordinates for geo-spatial matching.

```

CREATE TABLE BloodBanks ( BankID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(150) NOT NULL,
  GovtRegistrationNo VARCHAR(100) NOT NULL UNIQUE,

```

```
ContactEmail VARCHAR(100) NOT NULL UNIQUE,  
PasswordHash VARCHAR(255) NOT NULL,  
ContactPhone VARCHAR(15) NOT NULL UNIQUE,  
Latitude DECIMAL(10,7),  
Longitude DECIMAL(10,7),  
City VARCHAR(100) NOT NULL,  
Address VARCHAR(255)  
);
```

Geospatial Query – Nearest Donation Drives

This is the most technically interesting query in the system. It uses MySQL's ST_Distance_Sphere function to calculate the great-circle distance between the user's geocoded city and every drive on record, returning the ten closest results.

```
SELECT DriveName, DriveDate, Address, StartTime, EndTime,  
       ST_Distance_Sphere(  
           POINT(Longitude, Latitude),  
           POINT(:lng, :lat)  
       ) AS DistanceMeters  
FROM DonationDrives  
ORDER BY DistanceMeters ASC  
LIMIT 10;
```

Query – User Authentication Lookup

During login, the backend retrieves the stored password hash for a given email address. The hash is then verified in Python using Passlib's bcrypt context.

```
SELECT UserID, FullName, PasswordHash  
FROM Users  
WHERE Email = :email;
```

Query – 56-Day Donation Eligibility Check

The background scheduler runs this query to identify donors who became eligible to donate again today. The DATEDIFF function computes the number of days between the current date and the donor's last donation.

```
SELECT FullName, Email  
FROM Users  
WHERE LastDonationDate IS NOT NULL  
AND DATEDIFF(CURRENT_DATE, LastDonationDate) = 56;
```

Insert Query – Adding a Blood Unit

When a blood bank administrator logs a new collection, this parameterised insert creates a record with an initial status of "Available."

```
INSERT INTO BloodInventory (BankID, BloodGroup, CollectionDate, Status)
VALUES (:bank_id, :blood_group, :collection_date, 'Available');
```

Insert Query – Registering a New User

The registration endpoint geocodes the user's city and hashes the password before executing this insert. Latitude and longitude are stored alongside the profile for future proximity features.

```
INSERT INTO Users (FullName, Email, PasswordHash, PhoneNumber,
                  BloodGroup, Latitude, Longitude, City)
VALUES (:name, :email, :password_hash, :phone,
        :blood_group, :lat, :lng, :city);
```

Insert Query – Submitting a Blood Request

When a patient requests blood, this parameterised insert creates a record linked to their UserID with an initial status of "Pending."

```
INSERT INTO BloodRequests (UserID, BloodGroup, UnitsRequired, Status)
VALUES (:user_id, :blood_group, :units, 'Pending');
```

Backend – FastAPI Endpoint (Nearest Drives)

This Python endpoint demonstrates the full flow: geocoding the user's input city, running the spatial query, formatting the results as a JSON response, and handling errors gracefully.

```
@app.get("/api/nearest-drives")
def get_nearest_drives(city: str):
    location = geocator.geocode(city, country_codes="IN")
    if not location:
        raise HTTPException(status_code=404,
                            detail="City not found.")
    user_lat = location.latitude
    user_lng = location.longitude

    with engine.connect() as connection:
        query = text("""
            SELECT DriveName, DriveDate, Address,
            StartTime, EndTime,
            ST_Distance_Sphere(
                point(Longitude, Latitude),
                point(:lng, :lat)
            ) AS DistanceMeters
            FROM DonationDrives
            ORDER BY DistanceMeters ASC
            LIMIT 10;
        """)
```

```
result = connection.execute(query,
    {"lat": user_lat, "lng": user_lng})
# ... format and return JSON response
```

Backend – Password Hashing Utilities

Passlib's CryptContext provides a clean abstraction over bcrypt. The two helper functions below are used during registration and login respectively.

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"],
    deprecated="auto")

def get_password_hash(password):
    return pwd_context.hash(password)

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password,
        hashed_password)
```

Frontend – Direct Axios Integration

Instead of a centralized fetch wrapper, the application utilizes [axios](#) directly within React components to handle HTTP requests. This allows for cleaner asynchronous code and built-in JSON parsing. Below is an example of how the patient portal fetches nearby drives:

```
import axios from 'axios';
const searchDrives = async (e) => {
    e.preventDefault();
    try {
        const response = await
    axios.get(`http://127.0.0.1:8000/api/nearest-drives?city=${city}`);
        if (response.data.status === 'success') {
            setDrives(response.data.data);
        }
    } catch (error) {
        setError(error.response?.data?.detail || "Failed to fetch drives.");
    }
};
```

Frontend – AuthContext (Session Persistence)

The AuthContext provider wraps the entire application and persists the logged-in user's identity in localStorage. This pattern avoids the need for a heavier state-management library while ensuring the session survives page refreshes.

```

const AuthContext = createContext(null);

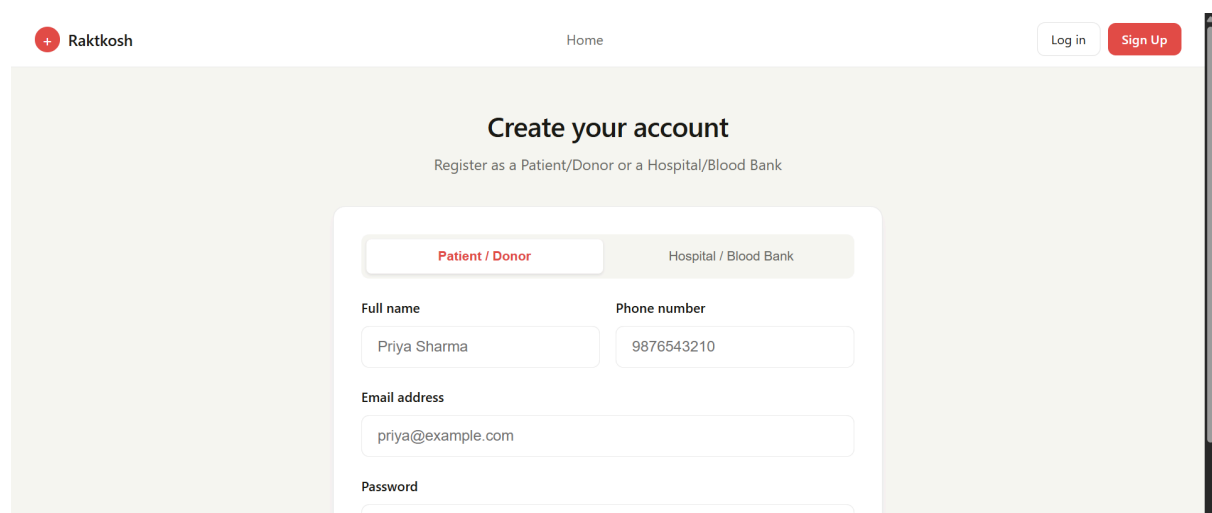
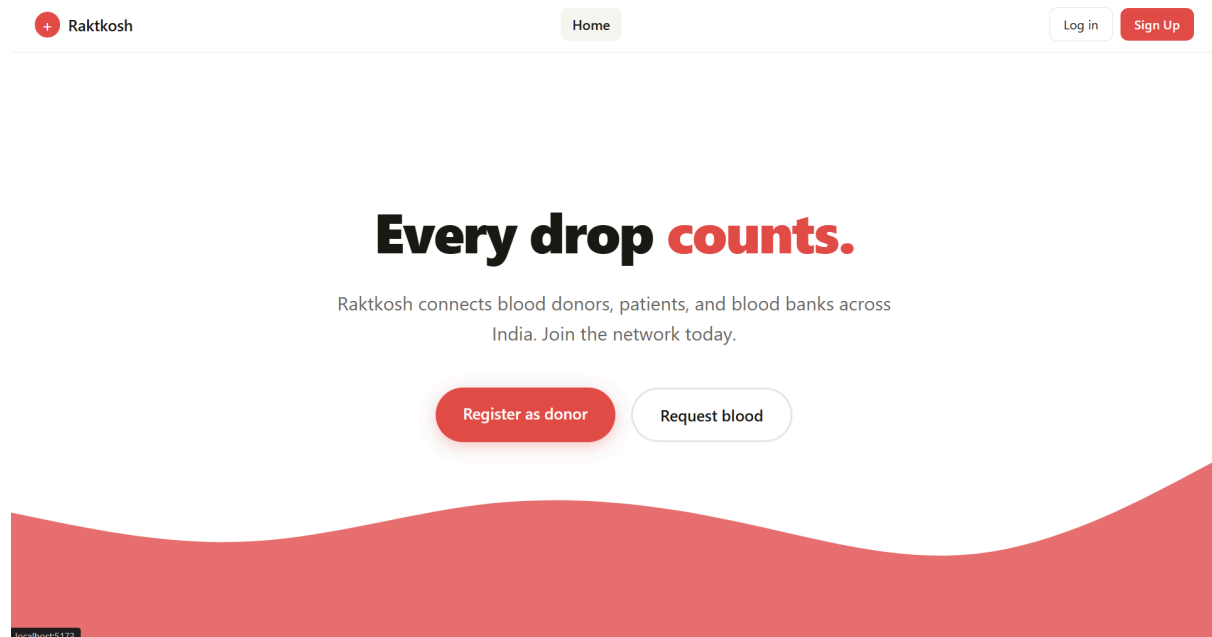
export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem("raktkosh_user");
    return saved ? JSON.parse(saved) : null;
  });

  function login(userData) {
    setUser(userData);
    localStorage.setItem("raktkosh_user",
      JSON.stringify(userData));
  }

  function logout() {
    setUser(null);
    localStorage.removeItem("raktkosh_user");
  }
  // ... Provider wraps children
}

```

6.5 Screens/forms (if any UI is built)



Raktkosh

Home

Log inSign Up

Patient / DonorHospital / Blood Bank

Hospital Name

City Central Hospital

Govt License Number

GOV-12345

Official Email

admin@hospital.com

Password

Create a strong password

Helpline Number

1800-123-456

City

Pune

Full Street Address

123 Main St, Near Park

Create account

Raktkosh

Home

Log inSign Up

Welcome back

Log in to your Raktkosh account

Email address

email@example.com

Password

Your password

Log in

Don't have an account? [Register](#)

Raktkosh

HomeFind BloodFind DrivesRequest Blood

Log out

Find Blood Near You

Search live hospital inventories sorted by distance.

Select Blood Group

A+

A-

B+

B-

AB+

AB-

O+

O-

Enter your city

Mumbai

Search

Mumbai Care Hospital

65 Main Street, Mumbai

9000000100

AB-

8.48 km away

Pune Lifeline Hospital

AB-

Raktkosh

Home

Find Blood

Find Drives

Request Blood

Log out

Find donation drives

See upcoming drives near you, sorted by distance

Enter your city

Delhi

Search drives

APR 30

New Delhi Mega Blood Camp 4

New Delhi

9:00:00 - 17:00:00

Upcoming

APR 22

New Delhi Mega Blood Camp 9

New Delhi

9:00:00 - 17:00:00

Upcoming

MAY

New Delhi Mega Blood Camp 14

Raktkosh

Home

Find Blood

Find Drives

Request Blood

Log out

Request blood

Submit a request — you'll be notified when a match is found

New blood request

Select blood group needed

A+

A-

B+

B-

AB+

AB-

O+

O-

Units required

1

Request for 3 units of B+ submitted.

Submit request

How this works

1. Select the blood group you need

2. Enter how many units you require

3. Submit your request

4. Admin portals instantly receive your live request

5. You get contacted when a match is found in your city

Raktkosh

Home

Pending Requests

Live Inventory

Add Blood

Schedule Drive

Log out

Live Blood Requests

Manage urgent patient requests (Sorted by closest to your hospital)

B+

3 Units Needed

8.5 km away

#16

Rohit Verma

Mumbai

8000000000

Requested: 2026-04-09 05:47:27

Contact Patient

A+

2 Units Needed

8.5 km away

#6

Anjali Singh

Mumbai

8000000005

Requested: 2026-04-09 04:51:55

Contact Patient

A+

2 Units Needed

8.5 km away

#11

Rahul Nair

AB-

1 Units Needed

8.5 km away

#1

Rohit Verma

Live Blood Inventory

Manage and track your hospital's available blood units

Filter by Blood Group

All Stock

A+

0 units

A-

0 units

B+

0 units

B-

0 units

AB+

0 units

AB-

0 units

O+

1 units

O-

0 units

BAG ID

BLOOD GROUP

COLLECTION DATE

STATUS

#25

O+

2026-02-20

Expired

Log Blood Units

Add newly collected blood bags to your live inventory

Add blood unit

Blood group — click to select

A+

A-

B+

B-

AB+

AB-

O+

O-

Collection date

dd-mm-yyyy



✓ A+ blood bag added successfully!

+ Add to inventory

Schedule Donation Drive

Organize a local blood camp to collect donations

Drive Name

e.g. City Center Summer Drive

City (For Location Tracking)

Pune

Drive Date

dd-mm-yyyy



Start Time

--:-- --



End Time

--:-- --



✓ Drive 'Mumbai Blood Drive' scheduled successfully!

Schedule Drive

CHAPTER 7

DATA FLOW RESULTS & DISCUSSIONS

7.1 Flow of Data across different Modules

Rakt Kosh is organised into three principal layers—the React frontend, the FastAPI backend, and the MySQL database—with an external geocoding service acting as a fourth collaborator for location-aware operations. This section traces how data moves through these modules for each major user workflow.

User Registration Flow

When a new donor fills out the registration form, data passes through four distinct modules before the process is complete.

Source	Action / Data Transformation	Destination
React (Register.jsx)	User fills form → JSON payload: {FullName, Email, Password, PhoneNumber, BloodGroup, City}	FastAPI Backend
FastAPI (main.py)	Validates blood group against allowed list; rejects invalid values with HTTP 400	Pydantic Model
FastAPI → Nominatim	Sends City string to Nominatim geocoder; receives {latitude, longitude}	Geopy Library
FastAPI → Passlib	Hashes plaintext Password using bcrypt; produces PasswordHash	Passlib/bcrypt
FastAPI → MySQL	INSERT into Users table with FullName, Email, PasswordHash, PhoneNumber, BloodGroup, Latitude, Longitude, City	MySQL Database

MySQL → FastAPI	Returns success or Duplicate Entry error	FastAPI Backend
FastAPI → React	JSON response: {status, message} → displayed as inline alert; redirects to Login on success	React (Register.jsx)

User Login Flow

The login flow involves the frontend, the API layer, and the database, with password verification handled by Passlib.

Source	Action / Data Transformation	Destination
React (Login.jsx)	User submits {Email, Password} as JSON POST	FastAPI Backend
FastAPI → MySQL	SELECT UserID, FullName, PasswordHash FROM Users WHERE Email = :email	MySQL Database
MySQL → FastAPI	Returns row with hashed password (or empty result)	FastAPI Backend
FastAPI → Passlib	verify_password(plain, stored_hash) → True/False	Passlib/bcrypt
FastAPI → React	JSON: {status, message, user_id} on success; HTTP 401 on failure	React (Login.jsx)
React (AuthContext)	Stores {userId, name} in localStorage; updates Context state	Browser Storage

Find Nearest Drives Flow

This is the most data-intensive flow, involving geocoding and a geospatial SQL query.

Source	Action / Data Transformation	Destination
React (FindDrives.jsx)	User enters city name → GET /api/nearest-drives?city=Mumbai	FastAPI Backend
FastAPI → Nominatim	Geocodes city string → returns {latitude, longitude}	Geopy Library
FastAPI → MySQL	SELECT ... ST_Distance_Sphere(POINT(Lng,Lat), POINT(:lng,:lat)) ... ORDER BY distance LIMIT 10	MySQL Database
MySQL → FastAPI	Returns up to 10 rows with DriveName, DriveDate, Address, Time, DistanceMeters	FastAPI Backend
FastAPI	Formats rows into JSON array; converts distance from metres to kilometres	Response Builder
FastAPI → React	JSON: {status, searched_location, data: [...drives]}	React (FindDrives.jsx)
React	Maps drive array into DriveCard components showing date, name, address, time, distance (km)	Browser DOM

Blood Request Flow

A logged-in patient submits a blood request, which flows from the frontend through validation to the database.

Source	Action / Data Transformation	Destination
React (FindBlood.jsx)	User selects blood group + units → JSON: {UserID (from AuthContext), BloodGroup, UnitsRequired}	FastAPI Backend
FastAPI (main.py)	Validates BloodGroup and checks UnitsRequired > 0	Pydantic Model
FastAPI → MySQL	INSERT INTO BloodRequests (UserID, BloodGroup, UnitsRequired, Status) VALUES (... , 'Pending')	MySQL Database
MySQL → FastAPI	Confirms insertion	FastAPI Backend
FastAPI → React	JSON: {status, message} → green success alert shown inline	React (FindBlood.jsx)

Add Inventory Flow (Dashboard)

Blood bank administrators log new collections through the dashboard's Add Blood Unit tab.

Source	Action / Data Transformation	Destination
React (AddInventory.jsx)	Admin selects blood group, date, bank ID → JSON: {BankID, BloodGroup, CollectionDate}	FastAPI Backend

FastAPI (main.py)	Validates BloodGroup against allowed list	Pydantic Model
FastAPI → MySQL	INSERT INTO BloodInventory (..., Status='Available')	MySQL Database
MySQL → FastAPI	Confirms insertion	FastAPI Backend
FastAPI → React	JSON: {status, message} → success alert; form resets for next entry	React (AddInventory.jsx)

Schedule Drive Flow (Dashboard)

New donation drives are created by administrators and geocoded before being stored.

Source	Action / Data Transformation	Destination
React (ScheduleDrive.jsx)	Admin fills DriveName, City, DriveDate, StartTime, EndTime, BankID → JSON POST	FastAPI Backend
FastAPI → Nominatim	Geocodes City → {latitude, longitude}	Geopy Library
FastAPI → MySQL	INSERT INTO DonationDrives with geocoded coordinates and Address = City	MySQL Database
MySQL → FastAPI	Confirms insertion	FastAPI Backend

FastAPI → React	JSON: {status, message} → success alert; form resets	React (ScheduleDrive.jsx)
-----------------	--	---------------------------

Background Automation Flow (56-Day Check)

This flow runs independently of any user interaction. The APScheduler triggers it at a configured interval.

Source	Action / Data Transformation	Destination
APScheduler	Fires check_donation_eligibility() on configured interval	FastAPI Backend
FastAPI → MySQL	SELECT FullName, Email FROM Users WHERE DATEDIFF(CURRENT_DATE, LastDonationDate) = 56	MySQL Database
MySQL → FastAPI	Returns list of eligible donors (name + email)	FastAPI Backend
FastAPI	For each donor: logs simulated email notification to console (production: SendGrid/Twilio)	Console / Email API

7.2 Output screenshots and Query Logic

Since the application runs locally and actual screenshots depend on the runtime environment, this section describes the expected output of each major operation alongside the underlying query logic that produces it. Each subsection pairs the user-visible result with the SQL and Python code that drives it.

Home Page Output

When the home page loads, the user is presented with a fluid, CSS-animated hero section featuring a dynamic wave visualization. This minimalist design focuses the user entirely on the platform's primary call-to-action buttons ("Register as donor" and "Request blood"). By avoiding static or heavily aggregated metric cards on the unauthenticated landing page, the system reduces unnecessary database load upon initial visitor entry and securely reserves heavy data operations for the authenticated, role-based dashboards.

Registration Output

On successful registration, the API returns the following JSON response:

```
// Backend response
{
  "status": "success",
  "message": "User Priya Sharma registered successfully!"
}
```

The frontend renders this as a green success alert inside the registration card, followed by an automatic redirect to the login page after two seconds. If the email already exists, the backend catches MySQL's Duplicate Entry error and returns:

```
{
  "detail": "Email or Phone Number already registered."
}
```

This is displayed as a red error alert within the same form, allowing the user to correct the input without losing their other field values.

Login Output

A successful login produces the following response, which the frontend parses to extract the user ID and display name:

```
// Backend response
{
  "status": "success",
  "message": "Welcome back, Priya Sharma!",
  "user_id": 42
}
```

The AuthContext stores {userId: 42, name: "Priya Sharma"} into localStorage. The Navbar immediately updates to show the Dashboard link and a Log Out button. On invalid

credentials, the API returns HTTP 401 with `{"detail": "Invalid email or password"}`, rendered as a red inline alert.

Nearest Drives Search Output

When a user searches for drives near “Mumbai,” the backend first geocodes the city (latitude: 19.076, longitude: 72.877), then executes the spatial query. A typical API response looks like:

```
{
  "status": "success",
  "searched_location": {
    "city": "Mumbai",
    "lat": 19.0759837,
    "lng": 72.8776559
  },
  "data": [
    {
      "name": "Andheri Blood Camp 2026",
      "date": "2026-04-15",
      "address": "Mumbai",
      "time": "09:00:00 - 17:00:00",
      "distance_km": 3.42
    },
    { ... } // up to 10 results
  ]
}
```

Each drive in the data array is rendered as a horizontal card. The date is split into a coloured box (month abbreviation on top, day number below). The distance in kilometres appears on the right edge of the card. If no drives exist near the searched city, the frontend displays a friendly empty-state card: “No drives found near Mumbai.”

Blood Request Output

When a logged-in user requests 2 units of O+ blood, the backend executes the insert and returns:

```
{
  "status": "success",
  "message": "Request for 2 units of O+ submitted."
}
```

The success message is displayed as a green alert inside the Find Blood card. If the user is not logged in, the form is gated by a persistent red banner: “Please log in to submit a request.” The submit button remains disabled in this state.

Add Inventory Output

When an administrator adds a B+ blood unit collected on 2026-04-08, the backend inserts the record and responds:

```
{
  "status": "success",
  "message": "B+ blood bag added successfully!"
}
```

The success alert appears inside the Add Blood Unit tab, and the form automatically resets the blood-group selection and date field so the administrator can quickly log the next unit.

Schedule Drive Output

After scheduling a drive named “Pune Health Camp” in Pune, the API geocodes the city and returns:

```
{
  "status": "success",
  "message": "Drive 'Pune Health Camp' scheduled successfully!"
}
```

The form resets all fields except the Bank ID, allowing the administrator to schedule multiple drives in quick succession.

Background 56-Day Check Output

The background scheduler produces console-level output. When a donor named Rohan Mehta (rohan@example.com) hits the 56-day mark, the server logs:

```
[SYSTEM] Running daily 56-day eligibility check...
[EMAIL SENT] To: rohan@example.com
| Message: Hi Rohan Mehta, it's been 56 days!
You can save another life today.
```

If no donors are eligible on a given day, the output is simply: “[SYSTEM] No users are hitting their 56-day mark today.” In a production environment, these console logs would be replaced by actual email or SMS API calls.

Explanation of queries/results

This section provides a detailed, line-by-line explanation of the most important SQL queries used in the project, describing what each clause does and what result set the database returns.

Geospatial Nearest-Drives Query

```
SELECT DriveName, DriveDate, Address, StartTime, EndTime,  
       ST_Distance_Sphere(  
         POINT(Longitude, Latitude),  
         POINT(:lng, :lat)  
       ) AS DistanceMeters  
FROM DonationDrives  
ORDER BY DistanceMeters ASC  
LIMIT 10;
```

This query is the backbone of the Find Drives feature. The `POINT(Longitude, Latitude)` expression constructs a geographic point from each drive's stored coordinates, while `POINT(:lng, :lat)` represents the user's searched location (obtained from Nominatim). MySQL's `ST_Distance_Sphere` function calculates the great-circle distance between these two points in metres, using the Haversine formula internally. The result is aliased as `DistanceMeters`, and the `ORDER BY` clause sorts all drives from nearest to farthest. The `LIMIT 10` cap ensures the response remains lightweight. The Python backend divides `DistanceMeters` by 1000 before returning the result to the frontend, converting it to kilometres rounded to two decimal places.

User Authentication Query

```
SELECT UserID, FullName, PasswordHash  
FROM Users  
WHERE Email = :email;
```

This query retrieves the minimum set of columns needed for authentication. The `Email` column has a unique constraint, so the result is either exactly one row or none. If no row is returned, the backend immediately raises an HTTP 401 error without revealing whether the email or the password was incorrect—a standard security practice that prevents user enumeration. If a row is found, the stored `PasswordHash` is passed to Passlib's `verify_password` function, which internally re-hashes the supplied plaintext password with the same bcrypt salt and compares the result. Only if both match does the server return the `UserID` and `FullName` to the frontend.

Donation Eligibility Check Query

```
SELECT FullName, Email  
FROM Users
```

```
WHERE LastDonationDate IS NOT NULL
AND DATEDIFF(CURRENT_DATE, LastDonationDate) = 56;
```

The DATEDIFF function returns the number of days between CURRENT_DATE and the donor's last recorded donation. The WHERE clause first filters out users who have never donated (LastDonationDate IS NOT NULL), then isolates those who hit exactly the 56-day mark today. The result set contains the names and email addresses of all newly eligible donors. The strictness of the equality check (= 56 rather than >= 56) ensures each donor is notified exactly once. If the scheduler missed a day, a production system would use >= 56 combined with a “notification sent” flag to avoid missed alerts.

Blood Inventory Insert

```
INSERT INTO BloodInventory
  (BankID, BloodGroup, CollectionDate, Status)
VALUES (:bank_id, :blood_group, :collection_date, 'Available');
```

This is a straightforward parameterised insert. The use of named parameters (:bank_id, :blood_group, :collection_date) rather than string concatenation protects against SQL injection attacks. The Status column is hardcoded to ‘Available’ at the time of insertion, and would later transition to ‘Issued’ when matched to a request or ‘Expired’ when the unit exceeds its 42-day shelf life. The BankID foreign key links the unit to the blood bank that collected it.

User Registration Insert

```
INSERT INTO Users
  (FullName, Email, PasswordHash, PhoneNumber,
   BloodGroup, Latitude, Longitude)
VALUES (:name, :email, :password_hash, :phone,
       :blood_group, :lat, :lng);
```

By the time this query executes, three transformations have already occurred in the Python layer: the plaintext password has been bcrypt-hashed into :password_hash, the city name has been geocoded into :lat and :lng, and the blood group has been validated against the allowed list. The unique constraints on Email and PhoneNumber mean that a duplicate insertion will raise a MySQL integrity error, which the backend catches and translates into a user-friendly message: “Email or Phone Number already registered.”

Blood Request Insert

```
INSERT INTO BloodRequests
  (UserID, BloodGroup, UnitsRequired, Status)
VALUES (:user_id, :blood_group, :units, 'Pending');
```

This query creates a new patient blood request. The UserID links the request to the authenticated user, enabling administrators to contact the requester. The Status is set to 'Pending' by default, and would be updated to 'Fulfilled' when inventory is allocated. The backend validates that UnitsRequired is at least 1 before executing the query, preventing zero-unit or negative requests from reaching the database.

7.3 Performance or observations

This section presents practical observations made during the development and testing of Rakt Kosh, covering response times, scalability considerations, and behavioural characteristics of each system module.

Backend API Response Times

During local development (FastAPI running on localhost with a MySQL instance on the same machine), the non-geocoded endpoints—login, add inventory, blood request—responded consistently within 20–60 milliseconds. The geocoded endpoints (registration, nearest drives, schedule drive) were noticeably slower, averaging 300–800 milliseconds per request, with the bulk of that latency attributable to the outbound HTTP call to the Nominatim geocoding service. On occasion, Nominatim responses exceeded one second, particularly during repeated rapid-fire requests, due to the service's rate-limiting policy (maximum one request per second for free-tier users).

Geospatial Query Performance

MySQL's ST_Distance_Sphere function performed well for the project's modest dataset (tens to hundreds of drives). However, this function cannot leverage spatial indexes because it operates on bare POINT() constructors rather than stored geometry columns. For datasets exceeding a few thousand rows, performance would degrade noticeably. The recommended production improvement would be to add a SPATIAL INDEX on a dedicated POINT geometry column and use ST_Within for a bounding-box pre-filter before applying the precise distance calculation to the shortlisted rows.

Frontend Rendering

React's virtual DOM diffing handled all UI updates smoothly, with no perceptible lag even when rendering ten drive cards simultaneously. The Vite development server provided sub-second hot module replacement, keeping the development feedback loop tight. The

overall bundle remains lightweight because the project avoids heavy component libraries—only React, React Router, and a minimal CSS file are shipped to the browser.

Authentication Observations

The bcrypt hashing operation (cost factor 12 by default in Passlib) adds approximately 200–400 milliseconds to the registration endpoint. While this is intentional—bcrypt’s computational cost is its primary defence against brute-force attacks—it is worth noting for throughput planning. The login path is slightly faster because verify is marginally less expensive than hash. The absence of JWT tokens means the current system has no concept of session expiry; a user remains logged in until they explicitly click Log Out or clear their browser’s localStorage.

Background Scheduler Observations

APScheduler’s interval-based trigger was configured at 10-second intervals for testing purposes, which allowed rapid verification of the 56-day check logic. In production, this would be set to run once daily (e.g., at midnight). The scheduler operates on a separate thread and does not block the FastAPI event loop, meaning API responsiveness is unaffected even when the eligibility check is in progress. A minor observation is that if the server is restarted and the scheduler misses its midnight window, the current equality check (`DATEDIFF = 56`) would silently skip donors who crossed the threshold during downtime. A production system should use a `>=` comparison paired with a boolean flag to guard against this.

Error Handling Observations

The backend’s error handling is functional but could be more granular. Currently, most exceptions are caught by a broad try/except block that returns HTTP 500 with the raw exception message. While this is helpful for debugging, exposing internal error details in production would be a security concern. The frontend handles errors gracefully through inline alert components, but does not implement retry logic or exponential backoff for failed network requests.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusions

Rakt Kosh demonstrates that a focused combination of modern web technologies can address a genuine public-health need—bridging the information gap between blood donors, patients in need, and the blood banks that serve them. The platform delivers a cohesive user experience spanning donor registration, secure authentication, proximity-based drive discovery, real-time inventory management, and automated eligibility tracking.

By leveraging React for a responsive single-page interface, FastAPI for a high-performance API layer, and MySQL's native spatial capabilities for distance calculations, the project strikes a practical balance between development speed and runtime efficiency. The architecture is deliberately modular: each endpoint, page, and data model is self-contained, making the system straightforward to extend, test, and maintain.

While the current implementation is scoped as a minimum viable product, it validates the core premise that technology can meaningfully improve blood-supply logistics. The insights gained during development—around geocoding reliability, background task orchestration, and client-side state management—provide a clear roadmap for the enhancements outlined in the next section.

8.2 Future Scope

Several avenues exist for extending Rakt Kosh into a production-grade platform:

- **JWT-Based Authentication and Role Management:** Replacing the current user-ID-based session with JSON Web Tokens would enable secure, stateless authentication with automatic expiry. Layering role-based access control on top would allow the system to distinguish between donors, patients, and blood bank administrators at the route level.
- **Real-Time Notifications:** Integrating a transactional email service such as SendGrid, or an SMS gateway like Twilio, would transform the 56-day eligibility check from a console log into an actual donor outreach channel. Push notifications via Firebase Cloud Messaging could further improve engagement.
- **Live Dashboard Metrics:** The home-page statistics are currently static placeholders. Connecting them to aggregate API endpoints that query actual

database counts would give users and administrators a truthful, real-time picture of platform activity.

- **Request-to-Inventory Matching:** Building an automated or semi-automated workflow that matches pending blood requests against available inventory—factoring in blood-group compatibility, expiry dates, and geographic proximity—would significantly reduce manual effort for bank administrators.
- **Active Blood Expiry Mutation:** Currently, the system uses dynamic frontend mathematical filtering and SQL Views to visually flag blood units as "Expired" once they pass the 42-day mark. In the future, we aim to implement a backend [APScheduler](#) cron job (similar to our 56-day donor check) that actively mutates the database [Status](#) column from 'Available' to 'Expired', which would allow for automated warning emails to hospital staff to minimize physical wastage.
- **Interactive Map Integration:** Embedding a Leaflet or Google Maps widget into the Find Drives page would let users visually explore nearby drives on a map, improving discoverability and overall user experience.
- **Production Deployment:** Containerising the application with Docker, deploying behind an Nginx reverse proxy with HTTPS, migrating to a managed database service, and replacing Nominatim with a commercial geocoding provider would prepare the platform for real-world traffic and reliability requirements.

REFERENCES

Journal / Conference Papers

- [1] Name 1 and Name 2, "Paper Title", Full Journal Name, volume no, page numbers, year
[2] Name 1 and Name 2, "Paper Title", Proceedings of the International / National Conference on ___, Institution, Country, page numbers, month, year

Reference / Hand Books

- [1] Name 1, "Book Title", Name of Publisher, Edition, ISBN number

Web

- [1] Topic 1, website name (do not include long URL's)

Arrangement of contents

- [1] Cover page
- [2] Inner page (same as cover page)
- [3] Dedication (Optional)
- [4] Certificate from college
- [5] Acknowledgements
- [8] Abstract
- [9] Table of Contents
- [10] List of Tables
- [11] List of Figures
- [12] Chapters 1, 2, 3, 4, 5
- [13] References (follow IEEE format)